DESIGNING A UNIVERSAL INFORMATION LANGUAGE

OSCAR BENDER-STONE

ABSTRACT. This thesis proposes a universal information language called Welkin. We introduce its use cases, as well rigorously define its rules and syntax. We prove that the syntax is unambiguous, and that Welkin can express any concept and proof recognized by some computer program. We conclude with future work, including standardization and revision management.

CONTENTS

1. Introduction . . . 1
2. Language Design . . . 4
   2.1. General Requirements . . . 4
   2.2. Units . . . 5
   2.3. Mechanizing Information . . . 6
   2.4. Base Operations . . . 6
3. Foundations . . . 7
   3.1. Words and Handles . . . 8
   3.2. Units . . . 8
   3.3. Turing Completeness . . . 11
   3.4. Queries and Information . . . 12
4. Syntax . . . 13
   4.1. Encoding . . . 13
   4.2. Invertible Syntax Descriptions . . . 15
   4.3. Strings and IDs . . . 16
   4.4. The Welkin Grammar . . . 16
   4.5. Validation and Transformations . . . 17
   4.6. Proof of Unambiguity . . . 18
5. Metatheory . . . 23
   5.1. Artëmov's Selector Proofs . . . 24
   5.2. Serial-Soundness and Reliable Proofs . . . 25
   5.3. Proof Completeness . . . 27
6. Conclusion . . . 28
   6.1. Comparisons to Existing Approaches . . . 29
   6.2. Future Work . . . 29
References . . . 31

DRAFT

1. INTRODUCTION

Information Management is the study of systematically storing and organizing data. Since the beginning of humanity, there has been a massive influx of data, including physical and digital forms. Not a single person understands all of it. For effective use, raw databases are increasingly structured into organized *information bases* to prove context surrounding the source and nature of data. The conceptual foundation for this process often traces back to Ackoff [Ack89]. His data-to-wisdom hierarchy posits that information is the process of transforming raw data into structured content. In turn, this content can be used to answer user queries. However, the term "information"

This thesis used Google Gemini (https://gemini.google.com/) for the following tasks: reviewing the iterature, discussing conjectures, clarifying new concepts, evaluating new language features, and creating templates for tables and figures. All ideas presented come from the author.

has not been standardized in the literature and is used with distinct meanings. For example, Buckland [Buc91] recognizes three different ways information is interpreted: as a *process*, as *knowledge*, and as *a thing*. While each of these are important, Buckland emphasizes that to *use* information, one must work with information as a thing—a tangible record like a document or digital file. Another author, Floridi, builds upon this tangible idea by requiring data to be truthful [Flo10]. Information must not only be in concrete records, but also accurately represent a topic of interest. As shown in this small sample of authors, the word information has several meanings, and they are driven by the goal to organize data.

Despite efforts to understand information, there are ongoing problems in real world projects. One of these issues is the *depth* of domains. For example, at major retailers like Amazon, inventory and recommendation systems use immense amounts of data to power personalized recommendations to users [SL17]. Another issue is the immense *breadth* of different subjects. As a well known example, Wikipedia struggles to constantly update its vast and interconnected web of knowledge, as found in [Mes+15]. Along with the depth and breadth of different disciplines, standardization is complex. Most systems have their own unique data formats, so making formats that are **interoperable** is hard. (A format is **interoperable** if data can be transferred to different formats.) For instance, transferring information between two medical platforms is notoriously difficult, even if they store the *same* information about a patient [Rei17]. These problems continue to be a challenge in effectively utilizing frameworks for information.

In consideration of these issues, several solutions partially resolve some of them, but not completely. The following provides a small list of major existing solutions. Reading these is optional; they are not needed to understand the thesis. We revisit these points in the conclusion (Section 6.1).

- **Resource Description Framework (RDF):** Systems based on this technology serve as the primary implementations of an internet standard called the Web Ontology Language (OWL) [Hit+09]. Information is stored as triples $\text{subject} - \text{predicate} \rightarrow \text{object}$. From there, the language can enforce rules and ensure new relationships are derived *only* from from previous ones. OWL has been highly successful in providing a machine-readable format to websites across the internet, outlined in [Hit21]. However, despite their widespread adoption, RDF-based solutions struggle with extensive sources of data. Enforcing logical rules can be expensive [Zhu+23]. Additionally, because OWL treats missing sources of data as *unknown* rather than *false*, it often fails to resolve conflicts between two sources [Hit21].
- **Labeled Property Graphs (LPGs):** This is an architecture defined by the ability to store rich, internal metadata within its components. Building on a network, consisting of nodes and edges between them, LPGs enable both nodes (entities) and edges (relationships) to possess a set of key-value pairs known as *properties* [Ang18]. The internal metadata allows systems to efficiently traverse networks, as attributes can be filtered and weighed on a single query pass. Due to their operational efficiency, LPGs are the primary choice in large scale commercial products, particularly real-time fraud detection tools and recommendation systems [RWE15]. Although LPGs are widely used, they are siloed by the lack of standards, making interoperability between databases challeng-

DRAFT

ing. Furthermore, LPGs lack an extensive logical engine, resulting in external tooling or complex query scripts [RWE15].

- **Cyc:** Designed to encompass broad subjects, Cyc is the largest proprietary database of knowledge, or *knowledge base*, known to date [Len95]. It achieves reliable queries by providing a logical engine, based on millions of hard coded "common sense" rules. However, despite its impressive scope, it relies on human experts to manually encode every rule. This limits how well the database scales [Har07]. Another issue is the absence of an active open standard, a result of the publicly available version, OpenCyc, being discontinued in 2017 [KBp].
- **fcaR (Formal Concept Analysis in R):** This tool is a modern implementation for the field of Formal Concept Analysis (FCA). FCA studies relationships through *formal concepts*, defined as a pair: an *extension* and an *intension*. Concepts are studied through a *concept lattice*, a hierarchical data structure that maps objects to attributes in a fixed domain. A modern implementation of this system is fcaR, a program developed in the R programming language. Given a fixed domain, the program can automatically identify the underlying core concepts and derived concepts [MDV26]. While being highly effective in specialized domains, concept lattices can become impractically large as the number of attributes increases [HS13]. This makes it difficult to interpret results when applied to large-scale databases or when bridging disparate domains [HS13].

This thesis designs a language to resolve these issues for Information Management. I call this language **Welkin**, based on an old German word meaning cloud [Dic25]. This word represents the expansive nature of the language, able to express any concept and justification that can be processed through a computer program. Welkin has three design goals, listed in Table 1: to be **universal**, **scalable**, and **standardized**. This thesis focuses on proving it is universal, discussed in detail at Section 2. The latter goals, to be scalable and standardized, are partially met in certain parts of the thesis. However, due to time restrictions, these are not fully complete and left for a future work.

The fundamental building block of Welkin is based on **representations**, containing a relationship between a **sign** representing a **referent** in a **context**.[1] Contexts provide flexibility to define domain-specific terminology, and are provided rules inspired by McCarthy's work in AI systems [McC93]. Moreover, truth is defined relative to contexts and enables the user to recognize *under which conditions* a statement is true. In this way, the information base keeps theories with conflicting claims isolated, providing a clear separation from other contexts.

Building on representations, information is defined as a sequence of representations, with each step being valid within a context and a base set of rules. To make the information base scalable, the base operations are kept as simple as possible. However, they are proven to be maximally expressive in Theorem 3.3.1 and Section 5. The simplicity of the language aids in standardization as well. However, to keep this thesis self-contained, we will discuss most definitions and proofs at a high level.

[1]There are similarities with representations and Peirce's semiotics. This theory studies the relationship between a symbol, the object it represents, and the interpretation that provides that meaning [Atk23]. Our notion is different in that contexts are a generalized interpretant; refer to Section 2.2.

DRAFT

Goal	Name	Content
G1	**Universality**	The language must enable user-created parameters, whosse symbolic representation is accepted by a computer program. Any program must be definable in the language.
G2	**Scalability**	The information base must appropriately scale to broad representations of information. Local queries must be efficient. Additionally, any future optimizations must be expressible *inside* the language itself.
G3	**Standardization**	The language needs a rigorous and formal specification. Moreover, the bootstrap must be formalized. The grammar and bootstrap must be fixed to ensure complete forwards and backwards compatibility. Certificates must be reliably checked, with a transparent way to view every step of a proof or argument.

TABLE 1. Goals for the Welkin language.

DRAFT

This thesis is organized according to Table 2.

Section Number	Title	Description
Section 2	**Rationale**	Introduces Welkin at a high level, with guiding examples.
Section 3	**Foundations**	Introduces the core theory behind Welkin, which is the starting point to specify the user-facing language.
Section 4	**Syntax**	Provides the grammar and proof that it is unambiguous.
Section 5	**Metatheory**	Proves that Welkin can express any proof.
Section 6	**Conclusion**	Concludes with comparisons to existing theories and possible applications, particularly in programming languages and broader academic knowledge management.

TABLE 2. Organization for the thesis.

2. LANGUAGE DESIGN

In this section, we justify the design of Welkin.

2.1. General Requirements.

The main purpose of an information base is to store information and answer user queries based on available information. To provide an upper bound on our requirements, we require two important properties:

1) The information base must be mechanized, which means only "effective operations" are allowed. We make "effective operations" precise with the technical notion of *partial computable functions*. Partial computable functions are computer programs that can process a finite input, and that either stop on an input or continue forever. The bedrock for these functions is the notion of a *Turing machine*, which models every current form of computation. For more background, refer to [Tur36]. To obtain universality, we need to express *every* one of these machines.
2) The information base must provide a mechanism to *ground* symbols defined by users. This stems from the Symbol Grounding Problem, posed by Harnard [Har90]. To avoid storing physical objects or such, users should have a way to provide *meaning* to symbols, or a way to *represent* their topic of interest.

Combined together, we can now state universality as follows: an information base is universal *if it can represent anything representable by a computer program.* We will clarify how we define "representable" in the next section.

2.2. **Units.**

The first core concept in Welkin is the notion of a **handle**. Handles are left as free parameters in the theory[2] In an information base, handles are uniquely identified through their **ID**.

Example 2.2.1. Consider a business that distributes vehicle parts. The inventory could be could be recorded via handles, one for each category. One could be for `tires`, another for `wheels`, and so for. Each of these names are then *distinct*, ensuring one category does not interfere with another. Specific items can be given their own handles as well. △

Having unique indexes is only one important aspect of handles. Another is that they are precisely by their relationships. These are written as **representations** $a - c \to b$. This is read as: a **represents** b **in context** c. Here, we call a the **sign**, b the **referent**, and c the **context**. One can interpret this characterization as follows: handles are defined by how they are and are *not* restricted. This idea is directly inspired from the philosophical idea of arbitrary objects, developed by Fine [Fin85]. These representations can be refined and adjusted to a given study of interest.

Example 2.2.2. An animal taxonomy can be stored through representations. Here, we write $\mathrm{A} -$ for handles A, B if B is a special case of A. For example, $\mathrm{Mammal} - \mathrm{Taxonomy} \to \mathrm{Dog}$ and $\mathrm{Mammal} - \mathrm{Taxonomy} \to \mathrm{Cat}$. Adding the rule $\mathrm{Animal} - \mathrm{Taxonomy} \to \mathrm{Mammal}$ would allow simple deductions, such as $\mathrm{Animal} - \mathrm{Taxonomy} \to \mathrm{Dog}$. △

We represent handles in Welkin using an extended notion called a **unit**. A unit is either a handle, a finite collection of units, or a representation. Collections of units can have a uniquely generated name by the information base called a **block**.[3]

Units define their own **context**, which means the unit provides its own names and representations. Writing $a - c \to b$, therefore, only applies to the *fixed* context c.

[2]For mathematicians, this is analogous to how key notions are left undefined in Hilbert's formalism of geometry, or how a "set" is an undefined notion in set theory.

[3]In programming languages, this is analogous to an anonymous function.

DRAFT

Example 2.2.3. A business could represent their operations using a unit business that contains units for their workers and ledgers. This allows another unit, say business2, to contain its *own* label workers that is separate from the one in business. In addition to separate labels, these contexts can have *distinct* rules, such as those for how business operations are performed. △

2.3. **Mechanizing Information.**

From the notion of a unit, we practically define information on a unit v as a *unit that tracks reductions in* v. Formally, we define information as a unit containing a derivation ending in a unit containing v. Key to this definition is making it *context-based*, thereby enabling separate truth-management systems (those systems defined by some Turing machine). This notion corresponds to Burgin's idea of information as an *operator* that transforms a system, and is closely to Bateson's famous quote that "information is a difference that makes a difference" [Bat00]. Our practical distinction between information and knowledge is that we *use* information, but users can assert their own notions of these terms by creating restricted contexts. There are technical constructions to show that our definition is *as general* as possible (Definition 3.4.12).

An important rule pivotal to information is the following rule called **contextual lifting**:

> Suppose a represents b in context c, and p represents q in context a.
> Then, *within* context c, p represents q in context b.

Written in symbols: $a - c \to b$ and $p - a \to q$ implies $p - b \to q$ *within* c. Lifting is a notion closely tied to McCarthy's notion of lifting, but formulated with representations. The idea is, if a represents b in context c, then a is a **surrogate** for b relative to c. In other words, a is a *faithful* proxy to b: if there is a relationship in context a, then it will *also* be in context b. Specific examples can be found throughout the literature, including [BF95]. Additionally, lifting means that the definition of information is theoretically *as general* as possible. For details, consult Section 5.

2.4. **Base Operations.**

Now, units and information themselves could be expressed in infinitely many languages, with slightly different syntax or semantics. Welkin is carefully designed to be a *minimal* expression of these concepts, with minimal friction to express any other universal information base. The core rules are provided in Table 4. To express the syntax of the language, we write these in ASCII in `type-writer` font.[4] These include:

- In ASCII, `a - b -> c` for representations. These can be interpreted *as* rewrite rules, depending on the context. The shorthand `a --> b` is used for the overarching context. Moreovoer, `a <--> b` means `a --> b` and `b --> a`.
- Traditional braces `{ }` to denote closed definitions of contexts, inspired by the C programming language.
- Paths via dots `.` that is inspired by the Python programming language. Relative paths are denoted with multiple dots `...`, and absolute imports are prefixed

[4]This font is Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

DRAFT

with `@`. Subsets of units can be written via `u.{v, x}`, or even `u.{v --> x}` to refer to a subset of representations.

- Imports are done through `@u`, which takes all units in `u` and puts them into the current scope. In other words, the implementation *implicitly* adds denotations `v <--> a.v` for each unit `v` in `u`.
- Selecting specific nested units can be done via `u.{v, x}` or, equivalently, `u.{v, x}`.
- Imports can be *negated* via the notation `{g, ~u}`. This is an uncommon feature of most programming languages, appearing primarily in Haskell and CSS.
- Comments *are* strings can be treated as any unit. No comments need to be removed in the files and can *enhance* the study of new subjects.[5] These units can *then* be analyzed by others to promote translations to other human languages, or be studied through the *overarching relationships* in the units.

In general, the minimal restricted keywords is crucial for providing support for other languages. An implementation of Welkin will contain a small section of ASCII encoding for easier standardization, but the rest of the program can be done *entirely* in the user's native language. This is a novel feature in most programming languages, which are either predominantly English or are fine tuned to a specific human language.

3. Foundations

This section discuss the foundations of Welkin, outlined in Table 3.

Section Number	Title	Description
Section 3.1	**Words and Handles**	Introduces binary words and handles.
Section 3.2	**Units**	Recursively defines units Definition 3.2.8 as their rules (Table 4).
Section 3.3	**Turing Completeness**	Proves that there is a natural correspondence between untis in Welkin and Turing machines.
Section 3.4	**Queries and Information**	Defines queries (Definition 3.4.11) and information (Definition 3.4.12)

Table 3. Overview of Section 4.

For this thesis, definitions and proofs will be given at a high level. Additionally, we adopt several conventions:

- Each **Definition** and **Remark** ends with a triangle (△).
- Each *Proof* ends with a square (□).
- We frequently abbreviate "if and only if" as "iff".

[5]Contexts can mimic regular comments, but Welkin aims to be inspectable by users. Encapsulation or private information can be enforced through *rejecting* specific contexts or only accepting certain ones.

DRAFT

3.1. **Words and Handles.**

For high-level notation, we write $a \equiv b$ to mean that a is definitionally equivalent to b. High level notation will *not* be stated in the syntax unless where noted. We will be pedantic in most notation for complete precision. Moreover, implementation details are outside the scope of this thesis. This includes, e.g., practically restricting user input by fixed upper bounds.

Definition 3.1.1. A **bit** is either of the symbols 0 or 1. A **binary word** is either the symbol ε (the **empty word**), and if w is a finite binary word, so are $w0$ and $w1$, where the juxtaposition of symbols denote **concatenation**. Nothing else is a word. $\triangle$

We also require a notion of equality on bits. To ensure this is constructive, we provide a *separate* definition of inequality as well. The latter provides an explicit certificate, a bit that shows how two words are distinct.

Definition 3.1.2. Equality $=_{\mathrm{W}}$ and inequality $\neq_{\mathrm{W}}$ on words w_1, w_2 is defined recursively and as nothing else:

- **Base case:** $\varepsilon = \varepsilon$.
- **Recursive step:** suppose $w_1 =_{\mathrm{W}} b_1 w_{1'}$ and $w_2 =_{\mathrm{W}} b_2 w_{2'}$, where b_1, b_2 as bits and $w_{1'}, w_{2'}$ are words. Then $w_1 =_{\mathrm{W}} w_2$ if and only if b_1, b_2 are both 0 or 1, and $w_1' =_{\mathrm{W}} w_2'$. Moreover, $w_1 \neq_{\mathrm{W}} w_2$ if and only if b_1 and b_2 are distinct bits or $w_1' \neq_{\mathrm{W}} w_2'$.

$\triangle$

Words alone do not carry meaning. Instead, meaning is provided through **handles**.

Definition 3.1.3. A **handle** is given by an **ID**, which is a word. The interpretation of a given handle is a free parameter, and therefore outside the scope of this language. $\triangle$

Because handles act as free parameters, we work with them through truth. Based on this, the rest of the thesis will abstract away general units *as* sets of axioms.

Now, we can define equality and inequality between handles. We will reserve $=$ and $\neq$ for handles, respectively.

Definition 3.1.4. Consider two handles h_1, h_2, and let ID_1 and ID_2 be their respective IDs. Then:

- h_1 is **equal** to h_2, written $h_1 = h_2$, if and only if $\mathrm{ID}_1 =_{\mathrm{W}} \mathrm{ID}_2$.
- h_1 is **not equal** to h_2, written $h_1 \neq h_2$, if and only if $\mathrm{ID}_1 \neq_{\mathrm{W}} \mathrm{ID}_2$.

$\triangle$

3.2. **Units.**

Now we can define units. Roughly, units are finite combinations of handles and representations. We present the complete definition as follows.

DRAFT

Definition 3.2.5. A **unit** is defined recursively as follows and nothing else:

- **Base case:**
  - The symbol $\{\}$, the **empty block**.
  - A handle, defined in Definition 3.1.6.
- **Recursive step:** let u, v, g be units and h a handle. Then any finite combination of the following are also units:
  - A **pair** $\{u, v\}$.
  - A **representation** or **arc** $a - c \to b$, where a is the **sign**, c is the **context**, and b is the **referent**. This is read as: a **represents** b **in context** c.

$\triangle$

We require more notation. First, we define $a \leftarrow c \to b$ to mean that both $a - c \to b$ and $a \leftarrow c - b$ hold. Second, we add a symbol $\preccurlyeq$ for **derives**, where $a \preccurlyeq g$ iff $a - b \to a$. The relation $a \preccurlyeq g$ is read as either "a is derived by g," or "g derives a".

Now we may introduce the rules on units. These are provided in Table 4, stated over stated over meta-variables a, b, c, d, g for units and h_1, h_2 for handles.

Rule	Name	Content
R1	**Internal Transitivity**	$a - c \to b$ and $b - c \to d$ imply $a - c \to d$
R2	**Pair Congruence**	If $a - c \to b$, then $\{a, d\} - c \to \{b, d\}$
R3	**Sign Congruence**	If $a - c \to b$, then $\{a - g \to d\} - c \to \{b - g \to d\}$
R4	**Context Congruence**	If $a - c \to b$, then $\{d - a \to g\} - c \to \{d - b \to g\}$
R5	**Referent Congruence**	If $a - c \to b$, then $\{d - g \to a\} - c \to \{d - g \to b\}$
R6	**Contextual Lifting**	$a - c \to b$ and $d - a \to g$ imply $\{d - b \to g\} \preccurlyeq c$
R7	**Refinement**	$\{a - c \to b, a - c \to d\} \longrightarrow \{a - c \to b\}$
R8	**Empty**	$\{g, \{\}\} \leftarrow c \to g$
R9	**Sign Absorption**	$\{\{\} - a \to b\} - c \to \{\}$
R10	**Context Absorption**	$\{a - \{\} \to b\} - c \to \{\}$
R11	**Referent Absorption**	$\{a - b \to \{\}\} - c \to \{\}$
R12	**Handle Equality**	If $h_1 = h_2$, then $h_1 \leftarrow c \to h_2$
R13	**Derivation**	$\{a - g \to a\} \leftarrow c \to \{\{g, a\} \longleftrightarrow g\}$
R14	**Idempotentcy**	$\{a, a\} \leftarrow c \to a$
R15	**Associativity**	$\{a, \{b, c\}\} \leftarrow c \to \{\{a, b\}, c\}$
R16	**Commutativity**	$\{a, b\} \leftarrow c \to \{b, a\}$

TABLE 4. Set of all valid rules in Welkin.

DRAFT

We review the utility of each rule. Note **R6** is the only rule *between* contexts; the rest is entirely user defined. Moreover, only **R1**, **R3**, and **R4** are needed for Turing completeness. However, the other rules are in place to help with organizing units as modules, as well as make it easier to use the language.

- **R1** ensures that each context is closed under transitivity. This makes it easier to reason *within* a context. Truth systems with non-transitive rules can be expressed through combining contexts.
- **R2-R5** enable the transfer of arcs onto pairs and other arcs.
- **R6** was previously discussed in Section 2.3. This provides a mechanism to test the relability of certain representations. That is, if a represents b in context c, then a is a **surrogate** for b. This means that all representations contained in a can be transferred onto b, *within* c.
- **R7** is explained as follows: suppose a is a unit, and in a block, a represents two other units b, d. Then this block *represents* $\{a \longrightarrow b\}$ and $\{a \longrightarrow d\}$, separately. This provides a mechanism to *refine* a general unit into a more specific one.
- **R8-R11** define the behavior of the empty unit $\{\}$, similar to the empty set. **R9** and the like specifically states that $\{\}$ contains *no* representations. Thus, if a representation has a component that is $\{\}$, then it carries no meaning. This provides a mechanism to *exclude* units in a context. This is useful for imposing invariants.
- **R12** enables equality in words and handles to pass through into representations. Besides this, note that equivalences on units are entirely user defined.
- **R13** characterizes $a \preccurlyeq g$ as $\{g, a\} \longleftrightarrow g$. This is closely related to the **semi-lattice** structure formed by units; refer to the point below.
- **R14-R16** ensure that information can be repeated and arranged in any order. Together with **R1**, this means that units have a **semi-lattice** structure, a term first introduced by Birkoff [Bir67]. Moreover, $\preccurlyeq$ is the corresponding partial order, which is a relation that is transitive ($x \preccurlyeq y$ and $y \preccurlyeq z$ entail $x \preccurlyeq z$) and reflexive ($x \preccurlyeq x$).

Remark 3.2.6. The rules in Table 4 use few, independent meta-variables, which may be equal or distinct. This phenomena is called "bundling", and it appears in the proof checker MetaMath Zero for first-order logic [Car20]. In first-order logic, the statement $\forall x. \forall y. x = y$ has two different meanings, depending on whether x and y denote the same meta-variable. Presenting these meanings requires *two* statements in first-order logic, and with slightly larger examples, this quickly explodes in size. All of this means that Welkin, similar to MetaMath Zero, achieves a significant level of compression. However, *direct* conversions to tools based on first-order logic may not be feasible. $\triangle$

In addition to Table 4, we need a form of **containment**.

Definition 3.2.7. Let u and c be units. Then u is **contained in** c, denoted $u \sqsubseteq c$, if either $u \equiv \{\}$, or for some unit v, $c \longleftrightarrow \{u, v\}$. $\triangle$

One can easily verify that $u \sqsubseteq v$ implies $u \preccurlyeq v$.

Finally, we add more notation. We write:

- $*\{a_1, ..., a_n\} - c \to b$ for $\{a_1 - c \to b, a_2 - c \to b, .., a_n - c \to b\}$.
- $a - c \to *\{b_1, ..., b_n\}$ for $\{a \longrightarrow b_1, a \longrightarrow b_2, .., a \longrightarrow b_n\}$.

Note that * is officially defined in the grammar, refer to Section 4.4.

DRAFT

3.3. **Turing Completeness.**

This section shows that Welkin is Turing complete. For background, there are many resources, e.g., [Sip13].

We want to demonstrate that every unit corresponds to some Turing machine, and vice versa. More precisely, we want to find a one-to-one mapping φ from units c to a Turing machine φ_c such that

$$a - c \to b \text{ if and only if } \varphi_c(\langle \varphi_a \rangle) = \langle \varphi_b \rangle. \qquad (\star)$$

Here, $\langle \varphi_a \rangle$ denotes a standard encoding of a Turing machine as a string (refer to [Sip13]). We also require that φ is surjective, i.e., each Turing machine T can be expressed as φ_u for some unit u. The purpose of φ is to provide a computational interpretation of units *as* programs, or operators. Note that handles can have implicit, user-defined meaning (outlined in Definition 3.1.6). Practically, we will maintain this interpretation, as representations are managed mechanically through truth.[6]

At this point, we are concerned with showing *some* φ exists, necessary to fulfill **G1**. We are not concerned about efficiency; this will be left for a future work, refer to Section 6. To construct φ, we want to create an embedding into the SK-combinator calculus. This is an equational theory, first developed by Schönfinkel [Sch24], and independently discovered by Curry [Cur30]. In this theory, a **combinator** is a higher-order function: a function that takes in other functions as inputs.[7] As a simplification, we present this calculus using a **reduction relation** instead of equality.

Definition 3.3.8. The SK-combinator calculus consists of the following:

- A **term** is defined recursively as either K or S, and if M, N are terms, so is (M) and their **application** MN.
- Evaluation: M **reduces** to N, written $M \Rightarrow_{\text{SK}} N$, if their equality can be deduced from the following axioms, defined over meta-variables A, B, P, Q:
  - **Base Axioms:**
    - $((KA)B) \Rightarrow_{\text{SK}} M$.
    - $(((SA)B)P) \Rightarrow_{\text{SK}} (AP)(BP)$.
  - **Transitivity:** if $A \Rightarrow_{\text{SK}} B$ and $B \Rightarrow_{\text{SK}} P$, then $A \Rightarrow_{\text{SK}} P$.
  - **Congruence:** if $A \Rightarrow_{\text{SK}} P$ and $B \Rightarrow_{\text{SK}} Q$, then $AB \Rightarrow_{\text{SK}} PQ$.

△

Now, Welkin can be embedded into this calculus. We discuss this embedding at a high level:

- Each unit Definition 3.2.8 can be built from handles, or by finite combinations of pairs and representations. These can be represented as combinators through suitable encodings of positive integers and finite tuples, respectively. For more details, refer to [Tur79].
- Each rule in Table 4 are themselves finite, ranging over a finite number of meta-variables. These, too, can be encoded as combinators through if/else statements.

[6] In connection to linguistics, this is the difference between a formal semantics, what is stated in the language, and pragmatics, the intension or purpose of a term.

[7] A remark for logicians: this calculus is extremely similar to a Hilbert-style proof system, with K and S corresponding to the rules $(\varphi \Rightarrow (\psi \Rightarrow \varphi))$ and $(\varphi \Rightarrow (\psi \Rightarrow \zeta)) \Rightarrow ((\varphi \Rightarrow \zeta) \Rightarrow (\psi \Rightarrow \zeta))$, respectively. This was one of Curry's insights in connecting logic to computation [Cur30].

DRAFT

It remains to show that every combinator is included in this embedding. For this, we prove the following. Our proof technique uses recursion within Welkin.

Theorem 3.3.9. *Let T be a term in the* SK*-combinator calculus. Then there is some unit u such that $T = \varphi_u$.*

Proof. We prove there is a unit L that can generate a unit u_T for each term of the SK calculus. To this end, we use K and S to represent the corresponding combinators in Definition 3.3.10, as well as M, N, P to represent meta-variables. With these, L is defined with exactly the following:

- $L - L \to {}^*\{K, S\}$: recall that this is equivalent to $L - L \to K$ and $L - L \to S$. These units represent that K and S, respectively, are terms.
- ${}^*\{M, N, P\} - L \to L$: recall that this is equivalent to $M - L \to L$, $N - L \to L$, and $P - L \to L$. Each of these represent meta-variables through **R1**. For example, $M - L \to K$ holds, but also $M - L \to S$. In other words, M can *potentially hold* K or S.
- $L - \{N - M \to L\}$: composition MN of two terms M, N is represented as $\{N - M \to L\}$, which itself is a term.
- $\{N - \{M - K \to L\} \to L\} - L \to M$: represents the reduction rule for K.
- $\{P - \{N - \{M - S \to L\} \to L\} \to L\} - L \to \{N - \{M - K \to L\} \to L\}$: represents the reduction rule for S.

We claim that L represents $\Rightarrow_{\text{SK}}$. First, to prove closure under composition, suppose $L - L \to A$ and $L - L \to B$. Then by **R1**, $M - L \to A$ and $N - L \to B$, so by **R3** and **R4**, $\{N - M \to L\} - L \to \{B - A \to L\}$. Another application of **R1** yields $L \longrightarrow \{B - A \to L\}$. The rules $L \longrightarrow K \mid S$ and $L - L \to \{N - M \to L\}$ are the only terms with sign L (besides L itself), so L contains *exactly* the terms in the SK-calculus.

Second, the base axioms for K and S are already included, and transitivity is provided by **R1**.

Finally, **R4** entails term congruence: if $M - L \to M'$, then $N - M \to L$ represents $N - M' \to L$. Because $\Rightarrow_{\text{SK}}$ and any term can be represented in L, this completes the proof. □

Taken together, we have completed one part of **G1**. The other part will be addressed in the next section.

3.4. **Queries and Information.**

By Theorem 3.3.1, any Turing machine can defined by a unit. Additionally, the construction *also* represents reductions of terms. This provides a ceiling on what queries we can express computably. For more details, refer to [HMU07].

Definition 3.4.10. Let c and q be units. A **query over** c is the following question: is q derived by c? △

Certain queries are easy, particular those restriicted to containment. Consider, for example, we immediately have that $p \sqsubseteq \{p, q\}$, hence $p \preccurlyeq c$. However, in general, this will be uncomputable, due to Theorem 3.3.1. In fact, this problem is is RE-complete, i.e., determining whether any Turing machine halts can be determined through a query in Welkin.

Although general queries are uncomputable, we can verify *certificates*. These provide a specific derivation, which demonstrates a query is true or false. In Welkin, this is exactly a derivation involving $q - c \to q$.

Definition 3.4.11. Let c and q be units.

- A **derivation over** c is a unit $\{u_1 - c \to u_2 - c \to ... - c \to u_n\}$ such that for each u_i, either a) u_i is contained in c, or b) u_i is an application of a rule in Table 4 from previous units $u_1, ..., u_j$.
- We say u **contains information** about a query q in context c if it contains a derivation that ends with the unit $\{q - c \to q\}$. Moreovoer, we say it **is information** if it only contains derivations ending in $\{q - c \to q\}$.

△

Our work in Section 5 proves that this definition is *complete*, or that *any* proof accepted by a Turing machine is accepted by Welkin. Note that this is possible thanks to contextual lifting (**R6**). Additionally, this enables proofs that certain queries are *not* possible. Nevertheless, this method is inherently incomplete, due to limitations in Turing machines. For more details, consult Section 5.

DRAFT

4. Syntax

This section examines the grammar for Welkin and provides validation rules. For specific sections, consult Table 5.

Section Number	Title	Description
Section 4.1	**Encoding**	Introduces the encoding for Welkin.
Section 4.2	**Invertible Syntax Descriptions**	Defines invertible syntax descriptions, the fundamental building blocks for the grammar.
Section 4.3	**Strings and IDs**	Defines the syntax for strings and IDs.
Section 4.4	**The Welkin Grammar**	Presentts the Welkin grammar, defining which strings are syntactically allowed.
Section 4.6	**Proof of Unambiguity**	Proves that the Welkin grammar is unambiguous.
Section 4.5	**Validation and Transforms**	Defines which strings are **valid**, as well as transforms on certain constructs.

Table 5. Overview of Section 4.

To make specific characters clearer, we write syntactic features in `type-writer font`.

4.1. Encoding.

Welkin uses ASCII as its base encoding. The term ASCII is slightly ambiguous, as there are subtly distinct variants. To address this problem, we formally define US-ASCII as

a standard version.[8] We define the encoding through **bytes**, which are words with of eight bits. We provide the codes for each character in decimal and hexadecimal. Note that the conversion into binary words, as directly used in Welkin, is straightforward.

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	|
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g			

TABLE 6. Printable US-ASCII codes and glyphs.

DRAFT

Definition 4.1.1. Welkin's encoding consists of **Printable US-ASCII**, listed in Table 6, as well as character `EOF`, with code 58. An **ASCII string** is the concatenation of bytes, each of which correspond to a character in the encoding. The last byte must end with the code for `EOF`. △

Moreover, we will need to define **character classes**, or important sets of characters. These are presented in Table 7, with each character separated by |. Note that strings *may* contain quotes prefixed with a slash \, refer to Listing 1.

[8]Note that this table *itself* is a representation, which represents glyphs with binary words.

Set Name	Characters
PRINTABLE	Listed in Table 6
WHITESPACE	\t | \r | Space
DELIMITER	{ | } | ' | " | . | ,
RESERVED	DELIMITER | * | @
SQ_CHAR	Any PRINTABLE character except ' and \
DQ_CHAR	Any PRINTABLE character except " and \
ID_CHAR	Any PRINTABLE character not in RESERVED or WHITESPACE

TABLE 7. Definitions of key character classes. Each class is denoted in UPPERCASE.

4.2. Invertible Syntax Descriptions.

To more easily describe the language, we provide basic building blocks. Our approach is based on *parser combinators*, which are functions that take other parsers as input. We augment these combinators using **invertible syntax descriptions**, first introduced in [RO10]. Here, an **invertible syntax description** is a pair of functions:

- A **Parser**, converts strings into an intermediate data structure.[9] We say the parser *consumes* a string.
- A **Printer**, the reverse of Parser.

Invertible syntax descriptions are used to enable easier formatting. This helps users more seamlessly switch between a string and the underlying unit. Specific details are provided in [RO10].

For our use case, we will need four invertible syntax descriptions. These are based on functions from `parsec`, a Haskell combinator parser library. Here, `A` and `B` denote meta-variables over syntax descriptions. Parsing `A` means applying the parser in `A`, and likewise, printing `A` means printing the corresponding rule. Additionally, `WS` is shorthand for `WHITESPACE` (Table 7).

- `A - seq -> B`:
  - **Parser:** Consumes a string based on `A`. immediately followed by the rule `B`. Notice that *no* `WS` characters are allowed.
  - **Printer:** prints the string `AB`.
- `A - lexeme -> B`:
  - **Parser:** Consumes a string based on `A`. then accepts any number of `WHITESPACE` characters, and finally consumes the input based on `B`.
  - **Printer:** first prints `A`, then zero or more `WS` characters, then `B`. white-spaces.
- `A - seq_many_till -> B`:
  - **Parser:** Consumes a string based on `A` zero more times, *until* the input is accepted by the rule `B`. Note that the end marker `B` supports unambiguous parsing.
  - **Printer:** prints `A*B`, where `A*` is zero or more instances of `A`.
- `A - lex_many_till -> B`:

[9]For computer scientists: this is an AST. Because this thesis aims to have a broader audience, we leave details to [RO10].

DRAFT

- **Parser:** Consumes a string based on `A` *and* optional whitespace zero more times, *until* the input is accepted by the rule `B`. Note that the end marker `B` supports unambiguous parsing.
- **Printer:** prints zero or more instances of `A` *with* optional whitespace, followed by `B`.

Additionally, the parsers above are allowed to have sets of characters called **tokens**. We assume these follow the "maximal munch" principle: the entire token must be consumed *before* processing other rules. This helps to simplify the grammar.

4.3. Strings and IDs.

Strings are defined in Listing 1, and they allow escaped single or double quotes. Note that, semantically, single and double quoted strings are equivalent, consult Section 4.5.

```
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR, "\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR, '\"'} - seq_many_till -> '"',
```

LISTING 1. Strings.

IDs are defined in Listing 2. These are special strings do not require quotes but forbid whitespace and certain characters. This comparison is reinforced in Section 4.5.

```
ID := ID_CHAR - seq_many_till -> *{RESERVED, WHITESPACE}
```

LISTING 2. Syntactic definition of an ID.

4.4. The Welkin Grammar.

Welkin's grammar is displayed in Listing 3, inspired by several languages.

- The binding operator `:=` is inspired by functional languages, emphasizing how Welkin is immutable. Moreover, choices can be written via `|`, which *themselves* can include definitions.
- The import system is inspired by Python. Additionally, `*u` is a concept borrowed from Python, which gets the contents of a tuple.[10]

[10]The grammar rule `trailer` is also inspired by Python. The Python grammar is available at: https://docs.python.org/3/reference/grammar.html

DRAFT

```
start := units - lexeme -> EOF,

units := "" | {unit - lexeme -> *{"", "," - lexeme -> units}},

unit := node - lexeme -> *{
  ":=" - lexeme -> choices,
  *{"", arc - lexeme -> chain}
},
choices := *{"", "|"} - lexeme -> choices_list,
choices_list := *{"",
  chain - lexeme -> *{"", "|" - lexeme -> choice_list}
},
chain := *{"", "*"} - seq -> path - lexeme -> *{"", arc - lexeme ->
chain},

arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -> "->",
other_arc := "<-" - seq -> node - seq -> *{"-", "->"},

node := *{"", "*"} - seq -> path,
path := *{"", modifiers} - seq -> trailer - seq -> *{"", graph},
modifiers := *{
  "@",
  "~" - seq -> *{"", "@"}
},
trailer := segment - seq -> *{"", "." - seq -> trailer},
segment := HANDLE - lexeme -> *{"", graph},
graph :=  "{" - lexeme -> units - lexeme -> "}",

HANDLE := ID | STRING,

STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR, "\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR, '\"'} - seq_many_till -> '"',

ID := ID_CHAR - seq_many_till -> *{RESERVED, WHITESPACE}
```

LISTING 3. The complete grammar for Welkin. This includes the rules `STRING` and `ID`, which are also provided in Listing 1 and Listing 2, respectively.

4.5. **Validation and Transformations.**

Due to time constraints, this section of the thesis is given at a higher level than intended. These details may change in the final revision of the language.

We say a string is **valid** if it is accepted by the grammar (Listing 3), and the following hold:

- The number of dots (relative import) used must not exceed the number of levels available.
- There must be at least one path that starts with `@h`, where `h` is some handle. This is used to name the module.
- For all paths that start with `@h1` and `@h2`, respectively, `h1` must be equal to `h2`.
- Every definition of a unit must only be stated once. This means `u := v, u:= w` is not allowed.
- Accessing a unit v from u requires that u has a closed definition containing v. This means `u := {v, w}, u.x` is not valid, and neither is `u.x` alone.

DRAFT

We leave error handling for future work, described in Section 6.

Additionally, we define transformation rules after parsing:

- Some node that starts with `@` is found. This marks the module name. Denote this name as m.
- If `\'` appears in a single quoted string, then this will be replaced by `'` in the final contents. For example, given a string `"John\'s dog"`, the contents would be `John's dog`.
- A similar rule applies for `"` but with `\"`.
- Double slashes `\\` are converted into one slash `\`.
- Single and double quoted strings are represent each other. For example, `'hello' <--> "hello"`. In fact, IDs are special cases of strings, so `hello <--> 'hello'`. Practically, the only difference is that units forbid `WHITESPACE` or `RESERVED` characters, but they can be written without quotes.
- Paths are resolved, determined where the unit originated. For example, in `a, c := {.a, b}`, the `.a` inside `c` would be expanded as `a`.
- Each `*u` for a handle or graph `u` expands to all the members in the enclosed unit. If `u` contains a at most one item or is a representation, then `*u` is expanded as `u`. Multiple `*`, such as `**u`, recursively expands the contents in a unit.
- Each definition `x := a1 | ... | an` expands to `x <--> {x - x -> a1, ..., b - x -> an}`. For each graph `a1`, the generated graph will contain `v - x -> v`, where `v` is a handle or representation in `a1`.
- Each `~x` expands to `x - g -> {}`, where `g` is the name where `~x` is placed. At the highest level, this would be the module name, m.

4.6. **Proof of Unambiguity.**

We show that, by construction, the combinators we used form an LL(1) grammar. These grammars have two desirable properties:

- If a string is accepted, it is parsed unambiguously.
- Efficient parsers can be easily and efficiently implemented [Aho+06].

Our approach is to based converting the combinators in [RO10] to an algebraic from, based on a paper by Edelmann, Hamza, and Kunčak [EHK20]. These authors define create an algebra on parsing combinators, used to define a combinator form of LL(1). In this algebra, two natural operations are disjunction $A \vee B$, which behaves exactly like $A \mid B$ in our notation, and $A \cdot B$, which represents concatenation. Additionally, the algebra includes rules with *exit variables*, or rules of the form $x \mapsto (A \cdot x) \cdot B$. Using recursion in x, these track when a parser still consumes an A, *or* encounters a B. For example, we can define a parser that takes in zero or more whitespaces: $\text{WS*} \equiv w \mapsto (\text{WS} \cdot w) \vee \varepsilon$. We heavily use exit variables in the conversions, provided in Table 8. For more details on the algebraic rules, refer to [EHK20].

DRAFT

Original	**Algebraic Form**
$A - \text{seq} \to B$	$A \cdot B$
$A - \text{lexeme} \to B$	$A \cdot (\text{WS}^* \cdot B)$
$A - \text{seq_many_till} \to B$	$x \mapsto (A \cdot x) \vee B$
$A - \text{lex_many_till} \to B$	$x \mapsto ((A \cdot \text{WS}^*) \cdot x) \vee B$
$C \coloneqq A \mid B$	$C \equiv A \vee B$
$^*\{A, B\}$	$A \vee B$
$^*\{\varepsilon, B\}$	$\varepsilon \vee B$

TABLE 8. Conversions of parser combinators into the algebra defined by [EHK20].

Next, we need to define properties of combinators, provided below. Note that $\varnothing$ denotes the empty set, $f \in F$ denotes that f is an element of F, and $F \cup E$ denotes the union of sets F and E.

DRAFT

Definition 4.6.2. ([EHK20]). Let A, B be meta-variables for parser combinators . Moreover, let t be a meta-variable for a simple combinator, one which consumes a single token. The following properties on parsing combinators are defined recursively:

- NULLABLE: returns a boolean, checking whether a combinator matches ε. We say a combinator is **nullable** if this check returns true.
  - $\text{NULLABLE}(\varepsilon) = \text{True}$.
  - $\text{NULLABLE}(t) = \text{False}$.
  - $\text{NULLABLE}(A \mid B) = \text{NULLABLE}(A) \text{ or } \text{NULLABLE}(B)$.
  - $\text{NULLABLE}(AB) = \text{NULLABLE}(A) \text{ and } \text{NULLABLE}(B)$.
- FIRST: gathers the first character in a combinator.
  - $\text{FIRST}(\varepsilon) = \varnothing$.
  - $\text{FIRST}(t) = \{t\}$.
  - $\text{FIRST}(A \mid B) = \text{FIRST}(A) \cup \text{FIRST}(B)$.
  - $\text{FIRST}(A \cdot B) =$
    $\text{FIRST}(A) \cup \text{FIRST}(B) \text{ if } \text{NULLABLE}(A) \text{ else } \text{FIRST}(A)$.
- SN-FOLLOW: this set stands for "should not follow". It is used to locally detect ambiguities.[11]
  - $\text{SN-FOLLOW}(\varepsilon) = \varnothing$
  - $\text{SN-FOLLOW}(t) = \varnothing$
  - $\text{SN-FOLLOW}(A \mid B) =$
    $\text{SN-FOLLOW}(A) \cup \text{SN-FOLLOW}(B) \text{ if } \text{NULLABLE}(A) \text{ else } \varnothing$.
  - $\text{SN-FOLLOW}(A \cdot B) = \text{SN-FOLLOW}(A) \cup \text{SN-FOLLOW}(B) \cup$
    $(\text{FIRST}(A) \text{ if } \text{NULLABLE}(B) \text{ else } \varnothing) \cup$
    $(\text{FIRST}(B) \text{ if } \text{NULLABLE}(A) \text{ else } \varnothing)$.

△

Using these properties, we are now ready to define $\text{LL}(1)$ grammars. Without loss of generality, we assume the set of parser combinators contains rules strictly of the form $A \mid B$, $A \cdot B$, or $x \mapsto (A \cdot x) \vee B$. We will implicitly use this form when examining the proof for the Welkin grammar.

Definition 4.6.3. ([Aho+06]). A set P of parsing combinators is $\text{LL}(1)$ if and only if, for all A and B:

- For any rule $A \mid B \in P$, at most A or B are nullable, and $\text{FIRST}(A)$ and $\text{FIRST}(B)$ are disjoint.
- For any rule $A \cdot B \in P$, $\text{SN-FOLLOW}(A) \cap \text{FIRST}(B) = \varnothing$. In other words, the set of characters that locally follow A must be disjoint with the characters starting in B.
- In any recursive expression $x \mapsto (A \cdot x) \vee B$, A is not nullable.

△

Finally, we conclude this section with the proof the Welkin grammar is $\text{LL}(1)$.

[11] A note for computer scientists: this is a *new* $\text{LL}(1)$ related set, replacing the usual FOLLOW set. This change is needed because parsing combinators are built bottom-up; they do not access a global parsing context. For details, consult [EHK20].

DRAFT

Theorem 4.6.4. *The Welkin grammar (provided in Listing 3) is* $\mathrm{LL}(1)$*, and is therefore unambiguous.*

Proof. We proceed by checking the necessary conditions. First, we need to confirm that in any rule of the form $A \mid B$, $*\{A, B\}$, at least A or B are not nullable. This is clear upon inspection.

Second, we check that, in all recursive rules of the form $x \mapsto (A \cdot x) \vee B$, A is not nullable. This, too, evidently holds: every rule in Section 4.4 based on $\mathrm{seq_many_till}$ and $\mathrm{lex_many_till}$ starts with at least one character. Thus, these rules have an A that is not nullable.

Third, we must confirm that certain pairs of sets are disjoint, based on Definition 4.6.15 and Table 8. More precisely:

- For each rule $A \mid B$ or $*\{A, B\}$, we need to confirm $\mathrm{FIRST}(A)$ and $\mathrm{FIRST}(B)$ are disjoint. We say this has **conflict type** First/First.
- For each rule $A - \mathrm{seq} \to B$ or $A - \mathrm{lexeme} \to B$, we need to confirm $\mathrm{SN\text{-}FOLLOW}(A)$ and $\mathrm{FIRST}(B)$ are disjoint. We say this has **conflict type** First/Follow. This order reflects that the FIRST sets must be calculated first, *then* the FOLLOW sets.

Note that a conflict type is refers to a *potential* conflict, a property that would invalidate being $\mathrm{LL}(1)$. Our last goal is to demonstrate these conflicts are *absent* in the grammar. This is readily verified in Table 10. To conserve space, each subrule is assigned an ID, and may be used in other subrules. We exclude easy cases, including `HANDLE`, or which start with a unique character. Moreover, any rule of the form `*{"", A}` clearly has disjoint FIRST sets, so we exclude most of these in the table.

With each of these properties satisfied, this proves that Listing 3 is $\mathrm{LL}(1)$. Thus, as all $\mathrm{LL}(1)$ grammars are uanmbiguous by construction, so is the Welkin grammar.

DRAFT

ID	Rule	Subrule
[1]	units	"," - lexeme -> units
[2]	units	"" | { unit - lexeme -> *{ "", [1] } }
[3]	unit	":=" - lexeme -> choices | *{ "", [7] }
[4]	choices	*{ "", "|" }
[5]	choice_list	chain - lexeme -> *{ "", [6] }
[6]	choice_list	"|" - lexeme -> choice_list
[7]	chain	arc - lexeme -> chain
[8]	arc	right_arc | other_arc
[9]	other_arc	*{"-", "->" }
[10]	node	*{ "", "*" }
[11]	path	*{ "", modifiers }
[12]	modifiers	*{"@", "~" - seq -> *{ "", "@" }}
[13]	trailer	"." - seq -> trailer
[14]	segment	*{ "", graph }

TABLE 9. IDs for each main subrule used in analysis.

DRAFT

ID	Conflict Type	Set One	Set Two	Intersection
[1]	First/ Follow	SN-FOLLOW(*{“”, [1]}) = { }, EOF }	FIRST([1]) = { , }	∅
[2]	First/ Follow	SN-FOLLOW([2]) = { }, EOF }	FIRST(unit) = { *, @, ~, ID, STRING }	∅
[3]	First/ First	FIRST(:=) = { := }	FIRST([7]) = { -, <- }	∅
[4]	First/ Follow	SN-FOLLOW([4]) = { *, ~, @, ID, STRING, ,, }, EOF, -, -> }	FIRST(|) = { | }	∅
[5]	First/ Follow	SN-FOLLOW(choice_list) = { ,, }, EOF }	FIRST([5]) = { *, ~, @, ID, STRING }	∅
[6]	First/ Follow	SN-FOLLOW(*{“”, [6]}) = { ,, }, EOF }	FIRST([6]) = { | }	∅
[7]	First/ Follow	SN-FOLLOW(*{“”, [7]}) = { |, ,, }, EOF }	FIRST([7]) = { -, <- }	∅
[8]	First/ First	FIRST(right_arc) = { - }	FIRST(other_arc) = { <- }	∅
[9]	First/ First	FIRST(-) = { - }	FIRST(->) = { -> }	∅
[10]	First/ Follow	SN-FOLLOW([10]) = { ~, @, ID, STRING, :=, -, <-, |, ,, }, EOF, -> }	FIRST(*) = { * }	∅
[11]	First/ Follow	SN-FOLLOW([11]) = { ID, STRING, :=, -, <-, |, ,, }, EOF, -> }	FIRST(modifiers) = { @, ~ }	∅
[12]	First/ First	FIRST(@) = { @ }	FIRST(~) = { ~ }	∅
[13]	First/ Follow	SN-FOLLOW(*{“”, [13]}) = { :=, -, <-, |, ,, }, EOF, -> }	FIRST([13]) = { . }	∅
[14]	First/ Follow	SN-FOLLOW([14]) = { ., :=, -, <-, |, ,, }, EOF, -> }	FIRST(graph) = { { }	∅

TABLE 10. Calculations for the main possible conflicts, along with the corresponding sets. Subrule IDs are listed in Table 9.

□

5. METATHEORY

This section proves that Welkin's definition of information (Definition 3.4.12) is as general as possible, outlined in Table 11. This proof is optional, and most readers are recommended to jump to Section 6. We assume the reader has background in first-

DRAFT

order logic, set theory, and the arithmetic hierarchy. For specific references, consult [Men09], [Mon73], and [Sim09], respectively.

Section Number	Title	Description
Section 5.1	**Artëmov's Selector Proofs**	Explains Artëmov's selector proofs. This includes his finitistic proof of Peano Arithmetic's consistency. Expands selector proofs for soundness.
Section 5.2	**Serial-Soundness and Reliable Proofs**	Expands selector proofs to **serial-soundness**, as well as **self-verifying** theories. Defines **reliable proofs** for more general theories.
Section 5.3	**Proof Completeness**	Proves that Welkin can accepts any proof from a provably sound RE theory.

TABLE 11. Overview of Section 4.

DRAFT

For simplicity, we will make our base theory Peano Arithmetic (PA). Recall that any unit is equivalent to a Turing machine (Section 3.3). Because any Turing machine can be expressed through a simple encoding in PA, so to can units, as well as the rules in Table 4. A future work will consider weaker theories that can embed Welkin. Additionally, we we will consider only RE first-order theories T. This is because any formal system must be defined with proofs accepted by some Turing machine. In turn, this Turing machine can be expressed as a first order theory.

As notation, we will write $T \vdash \varphi$ to mean that T entails φ. Logical implication is denoted by $\Rightarrow$.

5.1. **Artëmov's Selector Proofs.**

A major goal in Welkin is to express any representable notion, *including* any representabale proof. Proofs are a finite certificate that consists of a valid sequence of steps. Importantly, proofs can be verified using a *total* computable function, one that can check *any* given proof. Because of the requirement for computable checking, limitations in computable functions apply. These are illustrated by several impossibility theorems, including Gödel's infamous incompleteness theorems [Göd06].

However, previous impossibility results rely on proving strong properties of proofs. Artëmov's argues that weaker, constructive properties can be used instead. We provide a high-level definition of **serial-consistency** before proceeding onto the embedding in Welkin.

Definition 5.1.1. Suppose T is sufficiently strong to express its own proofs. Then T is **serial-consistent** if there is a **selector** for T. A **selector** is a computable function[12] s with two properties:

- s accepts any string that encodes a valid proof.
- if s accepts a proof, then it cannot contain a contradiction.

△

Artëmov overcomes Gödel's second incompleteness theorem. This theorem states that Peano Arithmetic PA, cannot prove its own consistency. Artëmov instead demonstrates that PA proves its own *serial-consistency*, as follows:

- He uses a weak meta-theory, in this case, Primitive Recursive Arithmetic (PRA). This theory is bounded by a weak principle of induction.
- Artëmov encodes all proofs as natural numbers in PRA; this theory is sufficiently strong *just* to encode all of these proofs.
- Finally, he uses a proof by induction, over encoded proofs in PRA, to show that PA proves each *individual* proof contains no contradictions. This is possible through partial truth predicates.

To be clear: Artëmov does *not* show that there is a *single* proof that PA is consistent, that works for all proofs. Instead, his method *takes* a proof as a parameter. There have been extensive discussion on the validity of this technique, and its acceptance by other logicians.[13]

Despite its initial controversy, Artëmov's techniques follow closely with several constructive schools, consult [Art24]. This thesis builds upon this result with a stronger property: **serial-soundness**. We will need a specific stronger version for our purposes.

5.2. Serial-Soundness and Reliable Proofs.

We now introduce a stronger version of **serial-soundness**. We will reuse the term **selector** for this stronger notion.

Definition 5.2.2. Let T be an extension of PA. Then T is **serial-sound** if there is a **selector** s. A **selector** is a total computable function of s constructed in PA, with inputs over proofs of T. This function must satisfy two properties:

- s accepts all proofs in T.
- Any proof p accepted by s must satisfy $\mathrm{Tr}_n(\langle s(p)\rangle)$, where Tr_n is the n-th partial truth predicate. For details, refer to [HP17].

Moreover, this selector must be proven *without* use the general Law of the Excluded Middle ($\varphi \vee \neg\varphi$), and without the Principle of Explosion ($\perp \Rightarrow \varphi$). In this case, we call s △

Remark 5.2.3. We need to remove the Law of the Excluded Middle to ensure the proof is completely constructive. Additionally, we must ensure Principle of Explosion is not used to falsely prove claims. △

[12]Technically, this is a primitive recursive function, but we will quickly generalize this to *any* computable function.

[13]The discussion is available online at: https://mathoverflow.net/questions/469247/situation-with-artemovs-paper.

DRAFT

Remark 5.2.4. We will briefly explain why, in the meta-theory, serial-soundness is enough to justify soundness. By way of contradiction, suppose T is a theory that is serial-sound but not sound.[14] Let s be a selector for T and let p be a proof in PA of the selector's correctness. Then, there must be some φ and some n such that $T \vdash \varphi$ but $\neg \mathrm{Tr}_n(\langle \varphi \rangle)$. However, for fixed n, p ensures that the selector correctly accepts the proof of φ in T. This yields a contradiction. Thus, T must be sound. △

Now we can introduce **self-verifying** theories.

Definition 5.2.5. Let T be an extension of PA. Then T is **(meta-)-self-verifying** if PA proves the following: T proves its own serial-soundness. △

Now, we can prove PA is self-verifying, using partial truth predicates. This is similar to the Artëmov's approach for serial-consistency. Note the limitations posed by Tarski's theorem. This says that a formal system cannot define its own truth predicate *at the object language* [Tar83]. In fact, this truth predicate is uncomputable, in general. We do not avoid Tarski's theorem, but we do approximate it *as* closely as Turing machines allow.

Additionally, there is an important property we will need.

Lemma 5.2.6. *Let T, T' be theories, with T being self-verifying. If T proves that T' is self-verifying, then so does* PA.

Proof. This corresponds to composing selector proofs. □

Remark 5.2.7. Welkin can prove Lemma 5.2.3 as well thanks to contextual lifting (**R6**). Let pa be a unit corresponding to PA. (This can be done through a Turing machine corresponding to PA and using Section 3.3.) Now, let T_1, T_2 be theories. △

From this, we define a T-**meta-proof** in PA as follows.

Definition 5.2.8. Let T be a self-verifying theory. Then a T-**meta-proof** in PA of φ, denoted $\mathrm{PA} \vdash_T \varphi$, is any proof of $\mathrm{PA} \vdash \langle T \vdash \varphi \rangle$. Here, $\langle T \vdash \varphi \rangle$ denotes the encoding of $T \vdash \varphi$ in PA. △

Note that the base theory T is important. This is because, two theories may be sound but prove φ and $\neg\varphi$, respectively. Additionally, we want to cover more general theories. A theory may be unsound yet prove *some* things correctly. For example, in Naive Set Theory (encoded in arithmetic), Russell's Paradox can be proven, but also true statements in ZFC, such as $\forall x. x \in X$ iff $x \in X$. Although this is a very simple example, this does raise the question: what *can* we reliably accept from a general theory? To generalize this example, we consider taking a *restriction* of the theory. If this theory is sound, provable in an extension that *is* self-verifying one, we can trust the specific claim from T'. Note that this applies to embeddings of languages as well. For simplicity, we will only discuss suitable subsets on T'. We provide the full definition below.

[14] Constructively, proof of contradiction may be used to prove a *negative* statement. Thus, our use of contradiction in this argument is permissible.

DRAFT

Definition 5.2.9. Let T' be a first order theory, and let φ be a sentence. A **reliable proof** of φ in T' is a pair (T'', p), where p is a proof that $T'' \vdash \varphi$, and $T'' \subseteq T'$ can be extended to a self-verifying theory. The set of **reliable proofs** for φ in T', denoted $\mathrm{Reliable}_{T'}(\varphi)$, is the set of pairs (T'', p_1, p_2), where T'' is described above, p_1 proves that T'' can extend to a self-verifying theory, and p_2 is a proof that $T'' \vdash \varphi$. △

Remark 5.2.10. Note that this definition is incomplete: in general, some sound subsets of T' will be excluded. We need to ensure that any theory used is *provably* sound, i.e., extends to a self-verifying theory. △

5.3. **Proof Completeness.**

To show that this covers *every* proof that can be computably recognized, we need to discuss recursive ordinals. For more background on ordinals, refer to [Mon73] and [Poh89]. For our purposes, we will only define specific properties needed for this thesis:

- Recursive ordinals, roughly, can be defined through a Turing machine. The full definition is in [Kle38].
- The limit of these ordinals is called the **Church-Kleene ordinal**, denoted ω_1^{CK}. This is the first non-recursive ordinal.
- Every RE first-order theory T has a recursive proof ordinal. Here, a proof ordinal is related to the strength of the theory. More details are available in [Kle38].
- There are existing encodings of ordinals into PA. For one approach, consult [Poh89].

Note that we can only reach so far with the successor stage. If we only permit unionis in the limit stage, we can only reach the Feferman–Schütte ordinal Γ_0, as proven in [Fef62]. This ordinal is related to predicative mathematics. However, the aim of Welkin is to be *as* expressive as possible, so we want to include *every* impredicative theory that we can express as well.

To reach these impredicative theories, we can utilize Artëmov's approach. Following his method, one can soundly jump from PA all the way to ZFC; this, too, can be done with partial truth predicates. One reason why ZFC is more powerful is because of the Axiom of Replacement, or even Comprehension. We extend this realization using hyperarithmetic sets, which are known to cover every recursive ordinal, proven in [Kle44]. To this end, our construction relies on a Simpson's principle of comprehension. This stems from Simpson's studies in Reverse Mathematics [Sim09]. The comprehension axiom states:

$$\exists X.\forall n.(n \in X \Leftrightarrow \Phi(n))$$

where $\Phi(n)$ is a predicate that is both Σ_1^1 and Π_1^1. For details, consult [Sim09].

We carry out this same construction for our use case. For simplicity, we will ensure that all constructed theories are self-verifying.

Let λ be a recursive limit ordinal, and consider all given theories $T_1, T_2, ..., T_\beta, ...$ with $\beta < \lambda$. Define

$$T' = \bigcup_{\beta<\lambda} T_\beta.$$

Note that, because the sequence above is indexed by recursive ordinals, T' is computable by some Turing machine. Moreover, T' is self-verifying: one can combine the

DRAFT

selectors for each T_j into a single selector. This requires dovetailing, which simulates each selector in parallel. The outputs of each selector can be incrementally added, ensuring that they are enumerated in the final selector.

Now, set $T_\lambda = T' \cup \mathrm{Comp}_\lambda(\Delta^1_1)$. Here, $\mathrm{Comp}_\lambda(\Delta^1_1)$ is an axiom schema over each predicate $\varphi(n)$ definable in T'. It states that:

$$\exists X_{\varphi,\lambda}.\forall n.\Big(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n)\Big)).$$

Constructing T_λ can be done with dovetailing, as described above. This ensures that each limit stage does *not* rely on non-constructive principles, such as Choice. Similarly, with the same technique, T_λ can be shown to be self-verifying.

Using the construction above, we can now prove the power of all meta-proofs in PA.

Theorem 5.3.11. *Through some self-verifying RE theory T, PA, equppied with all T-meta-proofs, can majorize any proof-theoretic ordinal. More precisely, for any recursive ordinal α, there is an RE theory T such that:*

- PA *meta-proves that T proves its own serial-soundness.*
- *T has a proof-theoretic ordinal greater than α.*

Proof. As previously mentioned, autonomous progressions are enough to reach the Feferman-Schütte ordinal Γ_0 [Fef62]. We may therefore focus on reaching higher ordinals, i.e., defining a suitable limit stage. For this, [Kle38] already proves that using hyper-arithmetic sets suffice, which is precisely done by the construction above. This completes the proof. □

Note, however, that detecting if an ordinal is recursive is undecidable. The upper bound is the best one can hope for, in general.

As an immediate corollary, we obtain the following.

Corollary 5.3.12. *Let T' be an RE theory, φ a sentence, and α a recursive ordinal. Then, for reliable proof (T'', p_1, p_2), there is a self-verifying extension T of T'', such that T has proof-theoretic ordinal greater than α.*

Combined with Corollary 5.3.5 and Remark 5.2.4, the previous results prove a key property of Welkin, stated as a major corollary.

Corollary 5.3.13. *Let pa be a unit corresponding to PA, as in Remark 5.2.4. Let T' be an RE theory and φ a sentence. Suppose $P \equiv (T'', p_1, p_2)$ be a reliable proof of φ in T'. Then, this proof in, written in T', is accepted by a derivation in pa. .*

6. Conclusion

This thesis proposed Welkin, a universal, formalized information language. Section 2 introduced the language design. Section 3 developed the recursive definition of a unit (Definition 3.2.8), rules on units Table 4, and queries (Definition 3.4.11), and information (Definition 3.4.12). This section also proved that every unit is equivalent to some Turing machine (Section 3.3). From there, Section 4 developed the grammar (Listing 3). The syntax was defined with invertible operations (Section 4.2), simultaneously providing *parsing* and *printing*. The main grammar was proven to be $\mathrm{LL}(1)$ Theorem 4.6.2, thereby showing it is unambiguous. Finally, Section 5 proved that the definition of information is complete Section 5.3, thereby completing **G1**.

DRAFT

The remaining sections provide comparisons to existing approaches, as well as future work.

6.1. Comparisons to Existing Approaches.

We compare from the solutions presented in the introduction (Section 1):

- **Resource Description Framework (RDF):** both Welkin and RDF use triple-based relationships. Many of the same ideas about representations are included in RDF. However, higher order representations are easier in the former, thanks to representations being units. Additionally, logical reasoning is significantlly more expressive and easier to use in Welkin, thanks to results like Section 3.3 and the work in Section 5. Managing diverse sources of information, as well as conflicts, is addressed with isolated contexts.
- **Labeled Property Graphs (LPGs):** this technology and Welkin are similarly flexible. Properties can be expressed through collections of arcs. The definition of containment (Definition 3.2.9) is straightforward, and it is likely to have an efficient implementation. However, in contrast to LPGs, Welkin has a more unified definition than LPGs, without affecting generality or specificity. Moreovoer, as with RDF, Welkin can easily express logical constraints.
- **Cyc:** both Welkin and Cyc support logical reasoning and rely on context-based reasoning. While Cyc currently encodes more rules, Welkin is more flexible and directly expressive for users. Additionally, Welkin is provided in an open formal specification, and is therefore not tied to proprietary details. A deeper comparison is limited precisely because Cyc is proprietary, in both its engine and hard-coded rules.
- **fcaR (Formal Concept Analysis in R):** Welkin uses a lattice through Table 4. This is similar to how Formal Concept Analysis (FCA) studies a concept lattice. Welkin can also embed core or derived concepts, due to the simplicity of Definition 3.2.9. Nevertheless, Welkin manages to enable a simpler lattice and still be completely universal. Certain proofs, when naively embedded, may be too large, but these can be adjusted through finding better proofs. Contexts also enable easier comparisons across multiple data sources.

6.2. Future Work.

The author has several future projects for Welkin in mind, particularly to reach goals **G2** and **G3**. These include:

- **Writing the Standard:** this would be a self contained document that describes. The Standard would *bootstrap* the language, providing step-by-step instructions on how to implement a Welkin information base. There would also be a core library that enables user created languages that convert *into* Welkin. For example, writing *in* Welkin, a user could define a grammar that takes their own syntax and *converts* it into units.
- **Creating an Efficient Implementation:** as the rules in Table 4, it is plausible that an efficient implementation would be possible. Moreovoer, optimizations could be expressed within Welkin itself, using handles for specific resources or operations, such as base operations in Python.
- **Query Solvers:** this would involve incorporating an information base with a program called a *solver*. Given a query, the solver would first confirms if there

is enough information in the database. If not, the solver runs its own internal methods and heuristics. Moreover, the solvers *themselves* can be specified in Welkin, making them easier to study and maintain.

- **Revision System:** because Welkin's grammar is relatively simple, a native revision system could be built as a core library. This could enable users to have greater control than existing tools. For instance, differences between versions could be easily analyzed *in* Welkin. This feature would be similar to a belief revision system, introduced in [AGM85].
- **Package Management:** building on revision systems, users could organize their Welkin code into packages. These would promote modular pieces of code, as well as simplify code distribution.

DRAFT

References

[Ack89] Ackoff, R L, From Data to Wisdom, Journal of Applied Systems Analysis **16** (1989) 3–9.

[Aho+06] Aho, Alfred V., Lam, Monica S., Sethi, Ravi, and Ullman, Jeffrey D., *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Addison Wesley, 2006.

[AGM85] Alchourrón, Carlos E., Gärdenfors, Peter, and Makinson, David, On the Logic of Theory Change: Partial Meet Contraction and Revision Functions, The Journal of Symbolic Logic **50** no. 2 (1985) 510–530.

[Ang18] Angles, Renzo, The Property Graph Database Model., in *AMW* (Olteanu, Dan and Poblete, Barbara (eds.), CEUR-WS.org.

[Art24] Artemov, Sergei, Serial properties, selector proofs and the provability of consistency, Journal of Logic and Computation **35** no. 3 (2024) exae34.

[Atk23] Atkin, Albert, Peirce's Theory of Signs, Spring2023 ed., Metaphysics Research Lab, Stanford University, 2023.

[Bat00] Bateson, Gregory, *Steps to an ecology of mind*, University of Chicago Press ed., University of Chicago Press, Chicago, 2000.

[Bir67] Birkhoff, G., *Lattice Theory*, 3rd ed., American Mathematical Society, Providence, 1967.

[Buc91] Buckland, Michael K., Information as thing, Journal of the American Society for Information Science **42** no. 5 (1991) 351–360.

[BF95] Buvač, Saša, and Fikes, Richard, A declarative formalization of knowledge translation, in *Proceedings of the Fourth International Conference on Information and Knowledge Management*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 340–347.

[Car20] Carneiro, Mario, Metamath Zero: Designing a Theorem Prover Prover, Springer-Verlag, Bertinoro, Italy, pp. 71–88.

[Cur30] Curry, H. B., Grundlagen der Kombinatorischen Logik, American Journal of Mathematics **52** no. 3 (1930) 509–536.

[Dic25] Dictionary, Oxford English, welkin, n., Oxford University Press, 2025.

[EHK20] Edelmann, Romain, Hamza, Jad, and Kunčak, Viktor, Zippy LL(1) parsing with derivatives, in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, Association for Computing Machinery, London, UK, pp. 1036–1051.

[Fef62] Feferman, Solomon, Transfinite Recursive Progressions of Axiomatic Theories, Journal of Symbolic Logic **27** no. 3 (1962) 259–316.

[Fin85] Fine, Kit, *Reasoning with Arbitrary Objects*, Blackwell, New York, NY, USA, 1985.

[Flo10] Floridi, Luciano, *Information : a very short introduction*, Oxford University Press, Oxford ; New York, 2010.

[Göd06] Gödel, Kurt, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, Monatshefte für Mathematik **149** no. 1 (2006) 1–29.

[Har90] Harnad, Stevan, The Symbol Grounding Problem, Physica D **42** (1990) 335–346.

[Har07] Harrington, Brian, ASKNet: Automated Semantic Knowledge Network., in *AAAI*, AAAI Press, pp. 1862–1863.

[HS13] Hazarika, Shyamanta, and Sinha, Smriti, Formal concept analysis: current trends and directions, Artificial Intelligence Review **44** (2013) 1–40.

[Hit21] Hitzler, Pascal, A review of the semantic web field, Commun. ACM **64** no. 2 (2021) 76–83.

[Hit+09] Hitzler, Pascal, Krötzsch, Markus, Parsia, Bijan, Patel-Schneider, Peter F., and Rudolph, Sebastian, *OWL 2 Web Ontology Language Primer*, 2009.

[HMU07] Hopcroft, John E., Motwani, Rajeev, and Ullman, Jeffrey D., *Introduction to Automata Theory, Languages, and Computation*, Third, Pearson/Addison Wesley, Boston, 2007.

[HP17] Hájek, Petr, and Pudlák, Pavel, *Metamathematics of First-Order Arithmetic*, Perspectives in Logic, Cambridge University Press, 2017.

[KBp] KBpedia, OpenCyc Resources.

[Kle38] Kleene, S. C., On Notation for Ordinal Numbers, The Journal of Symbolic Logic **3** no. 4 (1938) 150–155.

[Kle44] Kleene, S. C., On the Forms of the Predicates in the Theory of Constructive Ordinals, American Journal of Mathematics **66** no. 1 (1944) 41–58.

DRAFT

[Len95] Lenat, Douglas B, CYC: A large-scale investment in knowledge infrastructure, Communications of the ACM **38** no. 11 (1995) 33–38.

[McC93] McCarthy, John, Notes on formalizing context, in *Proceedings of the 13th International Joint Conference on Artifical Intelligence - Volume 1*, Morgan Kaufmann Publishers Inc., Chambery, France, pp. 555–560.

[Men09] Mendelson, Elliott, *Introduction to Mathematical Logic*, 5th ed., Chapman & Hall/CRC, 2009.

[Mes+15] Mesgari, Mostafa, Okoli, Chitu, Mehdi, Mohamad, Nielsen, Finn Årup, and Lanamäki, Arto, ``The sum of all human knowledge'': A systematic review of scholarly research on the content of Wikipedia, Journal of the Association for Information Science and Technology **66** no. 2 (2015) 219–245.

[Mon73] Monk, J. Donald, Introduction to Set Theory, Journal of Symbolic Logic **38** no. 1 (1973) 151.

[MDV26] Mora, Angel, Dominguez, Jesus, and Villalon, Ana, fcaR: Formal Concept Analysis, 2026.

[Poh89] Pohlers, Wolfram, *Proof theory : an introduction / Wolfram Pohlers*, vol. 1407, 1989.

[Rei17] Reisman, Miriam, EHRs: The Challenge of Making Electronic Data Usable and Interoperable, P & T : a peer-reviewed journal for formulary management **42** (2017) 572–575.

[RO10] Rendel, Tillmann, and Ostermann, Klaus, Invertible syntax descriptions: unifying parsing and pretty printing, in *Proceedings of the Third ACM Haskell Symposium on Haskell*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 1–12.

[RWE15] Robinson, Ian, Webber, Jim, and Eifrem, Emil, *Graph Databases*, 2nd ed., O'Reilly Media, Inc., 2015.

[Sch24] Schönfinkel, M., Über die Bausteine der mathematischen Logik, Mathematische Annalen **92** (1924) 305–316.

[Sim09] Simpson, Stephen G., *Subsystems of Second Order Arithmetic*, 2nd ed., Perspectives in Logic, Cambridge University Press, 2009.

[Sip13] Sipser, Michael, *Introduction to the Theory of Computation*, Third, Course Technology, Boston, MA, 2013.

[SL17] Smith, Brent, and Linden, Greg, Two Decades of Recommender Systems at Amazon.com, IEEE Internet Computing **21** no. 3 (2017) 12–18.

[Tar83] Tarski, Alfred, The Concept of Truth in Formalized Languages, Hackett Publishing Co., Indianapolis, IN, 1983.

[Tur36] Turing, Alan M., On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society **2** no. 42 (1936) 230–265.

[Tur79] Turner, D. A., A new implementation technique for applicative languages, Software: Practice and Experience **9** no. 1 (1979) 31–49.

[Zhu+23] Zhu, Xixi, Liu, Bin, Zhu, Cheng, Ding, Zhaoyun, and Yao, Li, Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning, Mathematics **11** no. 3 (2023).

University of Colorado at Boulder, Boulder, CO
Email address: osbe6746@colorado.edu

DRAFT